dtos

response

PaginaResponseDTO<T>

private contenido: List<T>
private pagina: int
private tamanio: int
private totalElementos: long
private totalPaginas: int

public desde(pagina: Pagina<T>): PaginaResponseDTO<T>

importacion

dto

«record» DonanteImportDto

tipoPersona: String
tipoDoc: String
documento: String
nombreORazonSocial: String
email: String
telefono: String

DonanteFactory

public crearPersona(dto: DonanteImportDto): PersonaDonante

DonanteCsvRowParser

public parsear(columnas: String[], numeroLinea: int): DonanteImportDto

«crea»

ImportReport

private resultados: List<ImportFilaCSV>

public agregar(result: ImportFilaCSV): void
public totalCreados(): long
public totalActualizados(): long
public getErrores(): List<ImportFilaCSV>
public tieneErrores(): boolean
public totalProcesados(): int

0..*

ImportFilaCSV

private numeroLinea: int
private estado: EstadoImport
private email: String
private detalle: String

public creado(linea: int, email: String): ImportFilaCSV
public actualizado(linea: int, email: String): ImportFilaCSV
public error(linea: int, email: String, motivo: String): ImportFilaCSV
public esError(): boolean

«enum» EstadoImport

CREADO
ACTUALIZADO
ERROR

integracion

notificaciones

NotificacionTransportAmqp

private rabbitTemplate: RabbitTemplate

public enviar(solicitud: SolicitudNotificacionMensaje): void
public nombre(): String

NotificacionTransportRest

private restClient: RestClient

public enviar(solicitud: SolicitudNotificacionMensaje): void
public nombre(): String

«record» SolicitudNotificacionMensaje

destinatario: String
mensaje: String
medio: String

Se crea una sola implementación según
la property notificaciones.transporte:
rest (predeterminado) → POST /notificaciones (8084)
amqp → exchange donatrack.notificaciones,
routing key notificacion.enviar
(RabbitMQConfig solo se carga con amqp)

config

RabbitMQConfig

public EXCHANGE: String = "donatrack.notificaciones"
public ROUTING_KEY: String = "notificacion.enviar"
public QUEUE: String = "notificaciones.queue"

public notificacionesExchange(): TopicExchange
public notificacionesQueue(): Queue
public notificacionesBinding(notificacionesQueue: Queue, notificacionesExchange: TopicExchange): Binding
public jsonMessageConverter(): MessageConverter
public rabbitTemplate(connectionFactory: ConnectionFactory, jsonMessageConverter: MessageConverter): RabbitTemplate

desde()

services

Entrega 5: cada lote de 500 filas es una transacción. Busca los emails del lote en una consulta (obtenerPorEmails) y guarda con guardarTodos (inserts en batch de 50). Si el lote falla, lo reprocesa fila por fila para informar el error exacto.

CsvImportService

private BATCH_SIZE: int = 500
private donanteRepository: PersonaDonanteRepositoryInterface
private parser: DonanteCsvRowParser
private factory: DonanteFactory
private transaccion: TransactionOperations

public importar(csvStream: InputStream): ImportReport
private procesarLote(lote: List<DonanteImportDto>, report: ImportReport, lineaInicio: int): void
private procesarLoteEnUnaTransaccion(lote: List<DonanteImportDto>, lineaInicio: int): List<ImportFilaCSV>
private procesarFilaSolo(dto: DonanteImportDto, linea: int): ImportFilaCSV
private actualizar(existente: PersonaDonante, dto: DonanteImportDto): void
private tieneTelefono(persona: PersonaDonante, telefono: String): boolean

«crea»

SegmentadorDonacionesService

private donacionesRepository: DonacionesRepositoryInterface
private donanteRepository: PersonaDonanteRepositoryInterface
private incentivosClient: IncentivosClient

public segmentar(carga: CargaDonacion): List<Donacion>
public cargarDonaciones(donaciones: List<Donacion>): void
private notificarIncentivos(idDonante: UUID, bienes: List<Bien>): void

AsignacionBatchService

private ESTADO_EN_DEPOSITO: String = "EN_DEPOSITO"
private donacionesRepository: DonacionesRepositoryInterface
private asignacionService: AsignacionDonacionesService
private candidatosRepository: CandidatosAsignacionRepositoryInterface
private entidadesRepository: EntidadesBeneficiariasRepositoryInterface

public ejecutarMatchmakingNocturno(): void «@Scheduled 0 30 3 * * *»
private calcularRanking(donacion: Donacion, entidades: List<EntidadBeneficiaria>): RankingPrecalculado

Entrega 5: PersonaDonanteService.eliminar y EntidadesBeneficiariasService.eliminarEntidad rechazan con 409 si borrado si hay donaciones (PersonaConDonacionesException / EntidadConDonacionesException).

DonacionService

private repositorio: DonacionesRepositoryInterface
private mapper: DonacionMapper
private asignacionService: AsignacionDonacionesService
private entidadesRepository: EntidadesBeneficiariasRepositoryInterface
private entidadesMapper: EntidadesBeneficiariasMapper
private donantesRepository: PersonaDonanteRepositoryInterface
private notificacionClient: NotificacionClient
private brokerClient: BrokerLogisticaClient
private validador: DonacionesValidator
private entidadesValidador: EntidadesBeneficiariasValidator
private candidatosRepository: CandidatosAsignacionRepositoryInterface

public obtenerDonaciones(filtro: FiltroDonaciones, pagina: SolicitudPagina): PaginaResponseDTO<DonacionResponseDTO>
public obtenerPorId(id: UUID): DonacionResponseDTO
public cambiarEstado(id: UUID, dto: CambioEstadoRequestDTO): void
public modificarBien(id: UUID, dto: BienRequestDTO): void
public obtenerCandidatos(id/donacion: UUID): CandidatosAsignacionResponseDTO
public obtenerCandidatosPendientes(): List<CandidatosPendientesResponseDTO>
public asignar(id/donacion: UUID, dto: AsignacionRequestDTO): void
public eliminar(id: UUID): void
private cargarEntidades(rankings: List<ListResultadoAsignacion>>) Map<UUID, EntidadBeneficiariaResponseDTO>
private mapearRanking(ranking: List<ResultadoAsignacion>, entidades: Map<UUID, EntidadBeneficiariaResponseDTO>): List<EntidadBeneficiariaResponseDTO>
private notificarDonacionEnEntrega(donacion: Donacion): void
private notificarAsignacion(donacion: Donacion, entidad: EntidadBeneficiaria): void

PlanificacionEntregasService

private ESTADO_ASIGNADA: String = "ASIGNACION_REALIZADA"
private donacionesRepository: DonacionesRepositoryInterface
private entidadesRepository: EntidadesBeneficiariasRepositoryInterface
private brokerClient: BrokerLogisticaClient
private validador: DonacionesValidator

public despacharUnaDonacion(donacionId: UUID): void
public despacharPendientesDeDia(): void «@Scheduled 0 0 4 * * *»
private validarYObtener(idsDonaciones: List<UUID>): Map<UUID, Donacion>
private tieneDireccionResolubleEnvio(donacion: Donacion): boolean
private entidadDe(donacion: Donacion): EntidadBeneficiaria
private aDonacionEnvio(donacion: Donacion): DonacionEnvio
private llamarABroker(proveedor: String, pedido: List<DonacionEnvio>): ResultadoEnvio

ReconciliacionRecepcionesService

private recepciones: RecepcionesEntidadRepositoryInterface

public reconciliar(): void «@Scheduled 0 0 2 * * *»

LogisticaEventosService

private donacionesRepositorio: DonacionesRepositoryInterface
private entidadesRepositorio: EntidadesBeneficiariasRepositoryInterface
private donanteRepositorio: PersonaDonanteRepositoryInterface
private notificacionClient: NotificacionClient
private incentivosClient: IncentivosClient

public procesarInicioRuta(dto: InicioRutaCallbackDTO): void
public procesarEntregaExitosa(dto: EntregaExitosaCallbackDTO): void
public procesarEntregaFallida(dto: EntregaFallidaCallbackDTO): void
private notificarInicioRutaEntidad(donacion: Donacion, urlMapa: String): void
private notificarInicioRutaDonante(donacion: Donacion, urlMapa: String): void
private notificarEntregaExitosaEntidad(donacion: Donacion, comprobante: String): void
private notificarEntregaExitosaDonante(donacion: Donacion, comprobante: String): void
private armarComprobante(dto: EntregaExitosaCallbackDTO): String
private notificarEntregaFallidaEntidad(donacion: Donacion, motivo: String, replanificable: boolean): void
private notificarEntregaFallidaDonante(donacion: Donacion, motivo: String, replanificable: boolean): void
private notificarEntregaFallidaAdmin(donacion: Donacion, motivo: String, replanificable: boolean): void
private obtenerDonacionOFalse(id: UUID): Donacion
private obtenerEmail(contactos: List<MedioDeContacto>): String

InactividadDonantesService

private DIAS_LIMITE_INACTIVIDAD: int = 20

private donanteRepository: PersonaDonanteRepositoryInterface
private notificacionClient: NotificacionClient

public notificarDonantesInactivos(): void «@Scheduled 0 0 0 * * *»
private notificarDonante(donante: PersonaDonante): void
private mapearTipoMedio(medio: MedioDeContacto): String

Las dependencias de cada service hacia las interfaces de interfaces repositorios figuran como atributos tipados y no se dibujan como flechas, para no saturar el diagrama. Los métodos «@Scheduled» indican la expresión cron del proceso nocturno; todos usan la zona America/Argentina/Buenos_Aires (FechaHoraArgentina.ZONA_ID), no la del servidor.

EstadosBeneficiarias

ASIGNADA
EN_DEPOSITO
ENTREGADA
CANCELADA

EstadosDonacion

PENDIENTE
ASIGNADA
EN_DEPOSITO
EN_CAMINO
ENTREGADA
CANCELADA

EstadosEntidad

ACTIVA
INACTIVA

util

FechaHoraArgentina

public ZONA_ID: String = "America/Argentina/Buenos_Aires"
public ZONA: ZoneId

public ahora(): LocalDateTime
public hoy(): LocalDate

NormalizadorEmail

public normalizar(email: String): String

«record» Pagina

contenido: List<T>
numero: int
tamanio: int
totalElementos: long

public totalPaginas(): int
public <R> map(transformacion: Function<T, R>): Pagina<R>

SolicitudPagina tiene validación en su constructor: page >= 0 y 1 <= size <= 100 (dte!); sin valor, 0 y 20); si no, PaginaInvalidaException → 400.
NormalizadorEmail: trim + NFC + minusculas (Locale.ROOT).
Es la forma con la que se guarda y se busca el email; lo usan PersonaValidator, PersonaDonanteRepository y CsvImportService.
Los services de donaciones y donantes devuelven PaginaResponseDTO.desde(Pagina) en los listados.

«record» SolicitudPagina

public TAMANIO_POR_DEFECTO: int = 20
public TAMANIO_MAXIMO: int = 100
numero: int
tamanio: int

public de(numero: Integer, tamanio: Integer): SolicitudPagina

interfaces

integracion

NotificacionTransport

public enviar(solicitud: SolicitudNotificacionMensaje): void
public nombre(): String

services

SegmentadorDonacionesServiceInterface

public segmentar(carga: CargaDonacion): List<Donacion>
public cargarDonaciones(donaciones: List<Donacion>): void

PersonaDonanteServiceInterface

public registrar(dto: PersonaDonanteRequestDTO): PersonaDonanteResponseDTO
public obtenerDonante(id: UUID): PersonaDonanteResponseDTO
public obtenerDonantes(estado: String, pagina: SolicitudPagina): PaginaResponseDTO<PersonaDonanteResponseDTO>
public cambiarEstado(id: UUID, dto: EstadoDonanteRequestDTO): void
public modificarContacto(id: UUID, dto: MedioDeContactoRequestDTO): void
public modificarRepresentante(id: UUID, dto: RepresentanteRequestDTO): void
public actualizar(id: UUID, dto: PersonaDonanteUpdateRequestDTO): PersonaDonanteResponseDTO
public eliminar(id: UUID): void

EntidadesBeneficiariasServiceInterface

public guardar(dto: EntidadBeneficiariaRequestDTO): EntidadBeneficiariaResponseDTO
public obtenerTodas(): List<EntidadBeneficiariaResponseDTO>
public obtenerPorId(id: UUID): EntidadBeneficiariaResponseDTO
public actualizar(id: UUID, dto: EntidadBeneficiariaRequestDTO): void
public eliminar(id: UUID): void
public agregarCampaniaAEntidad(idEntidad: UUID, dto: CampaniaRequestDTO): CampaniaResponseDTO
public agregarNecesidadACampania(idEntidad: UUID, campaniaId: UUID, dto: NecesidadRequestDTO): NecesidadResponseDTO
public obtenerNecesidadesDeCampania(entidadId: UUID, campaniaId: UUID): List<NecesidadResponseDTO>
public obtenerNecesidadesEntidad(id: UUID, campaniaId: UUID): NecesidadResponseDTO
public actualizarNecesidad(entidadId: UUID, campaniaId: UUID, necesidadId: UUID, dto: NecesidadRequestDTO): NecesidadResponseDTO
public eliminarNecesidad(entidadId: UUID, campaniaId: UUID, necesidadId: UUID): void

AlgoritmoAsignacionInterface

public evaluar(donacion: Donacion, entidades: List<EntidadBeneficiaria>): List<ResultadoAsignacion>

DonacionServiceInterface

public obtenerDonaciones(filtro: FiltroDonaciones, pagina: SolicitudPagina): PaginaResponseDTO<DonacionResponseDTO>
public obtenerPorId(id: UUID): DonacionResponseDTO
public cambiarEstado(id: UUID, dto: CambioEstadoRequestDTO): void
public modificarBien(id: UUID, dto: BienRequestDTO): void
public eliminar(id: UUID): void
public obtenerCandidatos(idDonacion: UUID): CandidatosAsignacionResponseDTO
public obtenerCandidatosPendientes(): List<CandidatosPendientesResponseDTO>
public asignar(idDonacion: UUID, dto: AsignacionRequestDTO): void

repositories

PersonaDonanteRepositoryInterface

public buscar(estado: String, pagina: SolicitudPagina): Pagina<PersonaDonante>
public obtenerInactivosDesde(limite: LocalDateTime): List<PersonaDonante>
public guardar(donante: PersonaDonante): void
public guardarTodos(donantes: Collection<PersonaDonante>): void
public modificarRepresentante(idDonante: UUID, representante: Representante): void
public existePorId(id: UUID): boolean
public obtenerPorEmail(email: String): PersonaDonante
public obtenerPorId(id: UUID): PersonaDonante
public obtenerPorEmails(emails: Collection<String>): Map<String, PersonaDonante>
public eliminar(id: UUID): void

EntidadesBeneficiariasRepositoryInterface

public guardar(entidad: EntidadBeneficiaria): void
public buscarTodas(): List<EntidadBeneficiaria>
public obtenerPorId(id: UUID): EntidadBeneficiaria
public obtenerPorIds(ids: Collection<UUID>): List<EntidadBeneficiaria>
public existePorId(id: UUID): boolean
public eliminar(id: UUID): void

DonacionesRepositoryInterface

public cargarDonaciones(donaciones: List<Donacion>): void
public buscarTodas(): List<Donacion>
public guardarAsignacion(donacion: Donacion): void
public obtenerPorEstado(estado: String): List<Donacion>
public buscar(filtro: FiltroDonaciones, pagina: SolicitudPagina): Pagina<Donacion>
public obtenerPorId(id: UUID): Donacion
public existenDelDonante(idDonante: UUID): boolean
public existeEnAlgunaEntidad(idEntidad: UUID): boolean
public eliminar(idDonacion: UUID): void

«record» FiltroDonaciones

estado: String
idDonante: UUID
subcategoria: String
idEntidad: UUID

public sinFiltros(): FiltroDonaciones

RecepcionesEntidadRepositoryInterface

public contarRecepcionesDesde(desde: LocalDate): Map<UUID, Long>
public contarRecepcionesDesde(idEntidad: UUID, desde: LocalDate): long
public reconciliar(): int

CandidatosAsignacionRepositoryInterface

public guardar(ranking: RankingPrecalculado): void
public obtenerPorDonacion(idDonacion: UUID): RankingPrecalculado
public obtenerTodos(): List<RankingPrecalculado>
public eliminar(idDonacion: UUID): void
public limpiar(): void

mappers

EntidadBeneficiariaMapper

private dependenciasMapper: PersonaDonanteMapper

public toModel(dto: EntidadBeneficiariaRequestDTO): EntidadBeneficiaria
public toDTO(entidad: EntidadBeneficiaria): EntidadBeneficiariaResponseDTO
public toCampaniaModel(dto: CampaniaRequestDTO, idEntidad: UUID): Campania
public toCampaniaDTO(campania: Campania): CampaniaResponseDTO
public toNecesidadModel(dto: NecesidadRequestDTO): Necesidad
public toNecesidadDTO(necesidad: Necesidad): NecesidadResponseDTO
public toNecesidadDTO(necesidad: Necesidad): NecesidadResponseDTO
private aplicarCamposComunes(necesidad: Necesidad, dto: NecesidadRequestDTO): void
private nombreTipo(necesidadODto: Object): String

PersonaDonanteMapper

public toModel(dto: PersonaDonanteRequestDTO): PersonaDonante
public toDTO(persona: PersonaDonante): PersonaDonanteResponseDTO
public toContacto(dto: MedioDeContactoRequestDTO): MedioDeContacto
public toContactoDTO(c: MedioDeContacto): MedioDeContactoResponseDTO
public toContactosDTO(contactos: List<MedioDeContacto>): List<MedioDeContactoResponseDTO>
public toDireccion(dto: DireccionRequestDTO): Direccion
public toLocalidad(dto: LocalidadRequestDTO): Localidad
public toDireccionDTO(dir: Direccion): DireccionResponseDTO
public toLocalidadDTO(localidad: Localidad): LocalidadResponseDTO
public toRepresentante(dto: RepresentanteRequestDTO): Representante
public toRepresentantes(dtos: List<RepresentanteRequestDTO>): List<Representante>
public toRepresentanteDTO(representante: Representante): RepresentanteResponseDTO

DonacionMapper

public toDTO(donacion: Donacion): DonacionResponseDTO
public toDTOS(set/donaciones: List<Donacion>): List<DonacionResponseDTO>
public toCargaDonacion(dto: DonacionRequestDTO): CargaDonacion
public toBien(dto: BienRequestDTO): Bien
private toBienesDTO(bienes: List<Bien>): List<BienResponseDTO>
private toBien(dto: DTO(dto: Bien): Bien<BienResponseDTO
private toHistorialDTO(historial: List<CambioEstado>): List<CambioEstadoResponseDTO>

validations

personas

PersonaValidator

private EMAIL: Pattern
private repositorio: PersonaDonanteRepositoryInterface

public validarEmail(email: String): void
public validarExistenciaPersona(id: UUID): void
public validarYObtenerPersona(id: UUID): PersonaDonante
public validarEsPersonaJuridica(id: UUID): void
public validarMedioContacto(medio: MedioDeContacto): void

entidades

EntidadesBeneficiariasValidator

private repositorio: EntidadesBeneficiariasRepositoryInterface

public validarExistenciaEntidad(id: UUID): void
public validarYObtenerEntidad(id: UUID): EntidadBeneficiaria
public validarYObtenerCampania(entidad: EntidadBeneficiaria, campaniaId: UUID): Campania
public validarYObtenerNecesidad(campania: Campania, necesidadId: UUID): Necesidad
public validarFechasCampania(inicio: LocalDate, fin: LocalDate): void

donaciones

DonacionesValidator

private repositorio: DonacionesRepositoryInterface

public validarTieneBienes(donacion: Donacion): void

repositories

PersonaDonanteRepository

private ORDEN_LISTADO: Sort = email.ASC
private jpa: PersonaDonanteJpaRepository

public guardar(personaDonante: PersonaDonante): void
public guardarTodos(donantes: Collection<PersonaDonante>): void
public obtenerPersona(id: UUID): PersonaDonante
public buscar(estado: String, pagina: SolicitudPagina): Pagina<PersonaDonante>
public obtenerInactivosDesde(limite: LocalDateTime): List<PersonaDonante>
public modificarRepresentante(idDonante: UUID, representante: Representante): void
public existePorId(id: UUID): boolean
public existePorEmail(email: String): boolean
public obtenerPorId(id: UUID): PersonaDonante
public obtenerPorEmail(email: String): PersonaDonante
public obtenerPorEmails(emails: Collection<String>): Map<String, PersonaDonante>
public eliminar(id: UUID): void

EntidadesBeneficiariasRepository

private jpa: EntidadBeneficiariaJpaRepository

public guardar(entidad: EntidadBeneficiaria): void
public buscarTodas(): List<EntidadBeneficiaria>
public obtenerPorId(id: UUID): EntidadBeneficiaria
public obtenerPorIds(ids: Collection<UUID>): List<EntidadBeneficiaria>
public existePorId(id: UUID): boolean
public eliminar(id: UUID): void

RecepcionesEntidadRepository

private jpa: RecepcionDiariaJpaRepository

public contarRecepcionesDesde(desde: LocalDate): Map<UUID, Long>
public contarRecepcionesDesde(idEntidad: UUID, desde: LocalDate): long
public reconciliar(): int

DonacionesRepository

private ORDEN_LISTADO: Sort = fechaDonacion DESC, id ASC
private jpa: DonacionJpaRepository
private recepciones: RecepcionDiariaJpaRepository

public cargarDonaciones(cargaDonaciones: List<Donacion>): void
public guardar(donacion: Donacion): void
public guardarAsignacion(donacion: Donacion): void
public obtenerPorEstado(estado: String): List<Donacion>
public buscar(filtro: FiltroDonaciones, pagina: SolicitudPagina): Pagina<Donacion>
public obtenerPorId(id: UUID): Donacion
public existenDelDonante(idDonante: UUID): boolean
public existeEnAlgunaEntidad(idEntidad: UUID): boolean
public eliminar(idDonacion: UUID): void
private ajustarRecepciones(idEntidad: UUID, fecha: LocalDate, delta: long): void
private especificacion(filtro: FiltroDonaciones): Specification<Donacion>

CandidatosAsignacionRepository

private jpa: RankingPrecalculadoJpaRepository

public guardar(ranking: RankingPrecalculado): void
public obtenerPorDonacion(idDonacion: UUID): RankingPrecalculado
public obtenerTodos(): List<RankingPrecalculado>
public eliminar(idDonacion: UUID): void
public limpiar(): void

Adapters sobre Spring Data JPA (base donaciones_db).
Cada método es @Transactional (las lecturas, readOnly).
Entrega 5:
- CandidatosAsignacionRepository persiste los rankings (tablas ranking_*); antes vivían en un ConcurrentHashMap.
- DonacionesRepository.guardarAsignacion guarda la donación y mueve el contador entidad_recepcion_diaria en la misma transacción (eliminar también lo descuenta).
- RecepcionesEntidadRepository.reconciliar vacía el contador y lo recalcula con un INSERT ... SELECT ... GROUP BY.
- Los listados (buscar) paginan y ordenan en la base.

jpa

PersonaDonanteJpaRepository

public findByEmail(email: String): Optional<PersonaDonante>
public existsByEmail(email: String): boolean
public findByEmailIn(emails: Collection<String>): List<PersonaDonante>
public findByEstadoNombre(estadoNombre: String, pagina: Pageable): Page<PersonaDonante>
public findByUltimaInteraccionAntesDe(limite: LocalDateTime): List<PersonaDonante>

EntidadesBeneficiariasJpaRepository

RecepcionDiariaJpaRepository

public totalesDesde(desde: LocalDate): List<TotalPorEntidad> «@Query»
public totalDesde(idEntidad: UUID, desde: LocalDate): long «@Query»
public sumar(idEntidad: UUID, fecha: LocalDate, delta: long): int «@Modifying»
public vaciar(): void «@Modifying nativa»
public recalcularDesdeDonaciones(): int «@Modifying nativa»

«proyeccion» RecepcionDiariaJpaRepository.TotalPorEntidad

public getIdEntidad(): UUID
public getTotal(): Long

DonacionJpaRepository

public findByEstadoNombre(estadoNombre: String): List<Donacion>
public existsByIdEntidadBeneficiaria(idEntidad: UUID): boolean
public existsByIdEntidadBeneficiariaAndEstadoNombre(idEntidad: UUID, estadoNombre: String): boolean
public buscarAsignacion(id: UUID): Optional<AsignacionActual> «@Query»

«proyeccion» DonacionJpaRepository.AsignacionActual

public getIdEntidadBeneficiaria(): UUID
public getFechaAsignacion(): LocalDate

RankingPrecalculadoJpaRepository

public vaciarCoincidencias(): void «@Modifying nativa»
public vaciarRankingSemanticos(): void «@Modifying nativa»
public vaciarRankingSubAlimentos(): void «@Modifying nativa»
public vaciarRankings(): void «@Modifying nativa»

Spring Data JPA

«externa» PersonaDonante, UUID
«externa» JpaRepository

«externa» EntidadBeneficiaria, UUID
«externa» JpaRepository

«externa» RecepcionDiariaEntidad, Clave
«externa» JpaRepository

«externa» Donacion
«externa» JpaSpecificationExecutor

«externa» Donacion, UUID
«externa» JpaRepository

«externa» RankingPrecalculado, UUID
«externa» JpaRepository

algoritmos

AlgoritmoCompatibilidadSemantica

protected calcularPuntaje(donacion: Donacion, entidad: EntidadBeneficiaria): double
protected incluirJuntaje: double): boolean

AlgoritmoPrioridadSubAlimentos

private MESES_TRIMESTRE: int = 3
private recepciones: RecepcionDiariaEntidadRepository

protected prepararPuntaje(donacion: Donacion, entidades: List<EntidadBeneficiaria>): ToDoubleFunction<EntidadBeneficiaria>
protected calcularPuntaje(donacion: Donacion, entidad: EntidadBeneficiaria): double
private mesesEnTrimestre(): LocalDate
private puntaje(donacionesRecibidas: long): double

«abstracta» AlgoritmoAsignacionBase

protected TOP_N: int = 10

public final evaluar(donacion: Donacion, entidades: List<EntidadBeneficiaria>): List<ResultadoAsignacion>
protected prepararPuntaje(donacion: Donacion, entidades: List<EntidadBeneficiaria>): ToDoubleFunction<EntidadBeneficiaria>
protected calcularPuntaje(donacion: Donacion, entidad: EntidadBeneficiaria): double
protected incluirJuntaje: double): boolean

AlgoritmoCompatibilidadSemantica

private algoritmoSemantico: AlgoritmoCompatibilidadSemantica
private algoritmoSubAlimentos: AlgoritmoPrioridadSubAlimentos
private entidadesRepository: EntidadesBeneficiariasRepositoryInterface

public generarRanking(donacion: Donacion): ResultadoMatchmaking
public generarRanking(donacion: Donacion, entidades List<EntidadBeneficiaria>): ResultadoMatchmaking
private intersectar(a: List<ResultadoAsignacion>, b: List<ResultadoAsignacion>): List<ResultadoAsignacion>

Template method: evaluar() llama una vez a prepararPuntaje() por ranking. SubAlimdidos lo redefine para leer en UNA consulta las recepciones del trimestre de todas las entidades (tabla entidad_recepcion_diaria, Entrega 5).

ResultadoMatchmaking

private coincidencias: List<ResultadoAsignacion>
private rankingSubAlimdidos: List<ResultadoAsignacion>

public huboCoincidencias(): boolean

clientes

IncentivosClient

private restClient: RestClient

public notificarDonacionRegistrada(donanteId: UUID, destinatario: String, medio: String, cantidadBienes: int, categoriasDonadas: List<String>): void
public notificarDonacionRegistrada(donanteId: UUID, destinatario: String, medio: String, cantidadBienes: int, categoriasDonadas: List<String>, fecha: LocalDateTime, entidadBeneficiaria: String): void
public notificarDonacionExitosa(donacion: Donacion, destinatario: String, medio: String): void
public notificarDonacionExitosa(donacion: Donacion, destinatario: String, medio: String, cantidadEventos: LocalDateTime): void
private notificarDonacionExitosa(donanteId: UUID, destinatario: String, medio: String, cantidadBienes: int, categoriasDonadas: List<String>, fecha: LocalDateTime, entidadBeneficiaria: String): void
private cantidadBienes(donacion: Donacion): int
private categoriasDonadas(donacion: Donacion): List<String>
private entidadBeneficiaria(donacion: Donacion): String

NotificacionClient

private transporte: NotificacionTransport

public registrarTransporte(Action): void «@PostConstruct»
public enviarNotificacion(destinatario: String, mensaje: String, medio: String): void

BrokerLogisticaClient

private restClient: RestClient

public solicitarEnvio(proveedor: String, donaciones: List<DonacionEnvio>): ResultadoEnvio

«record» EnvioAsignado

idDonacion: UUID
proveedor: String
idSeguimiento: String
fechaAsignacion: LocalDateTime

«record» ResultadoEnvio

proveedor: String
proveedoresDescartados: List<String>
envios: List<EnvioAsignado>

«record» DonacionEnvio

idDonacion: UUID
idEntidadBeneficiaria: UUID
nombreEntidad: String
direccionEntrega: DireccionEnvio

«record» DireccionEnvio

calle: String
numero: int
localidad: String
provincia: String
codigoPostal: String

«record» SolicitudEnvio

proveedor: String
donaciones: List<DonacionEnvio>

IncentivosClient → POST /api/incentivos/donanteId/donacion y /donacion-exitosa (servicio-incentivos, 8083)
BrokerLogisticaClient → POST /api/brokers/envios (servicio-broker-logistica, 8087)